

Spring Semester 2026 EE209 Final Exam

Pledging of No Cheating

Note: Please write down your student ID and name, sign on it (draw your signature), and date it. If you do not fill out this page, we won't grade your final exam.

저는 이번 시험을 치르면서 이 과목에서 금지한 어떤 부정행위도 저지르지 않을 것임을
서약합니다. 추후에 위반사항이 발견되었을 경우 합당한 모든 불이익을 감수하겠습니다.

I pledge that I will not participate in any activity of cheating disallowed by this course while taking this exam. I will assume full responsibility if any violation is found later.

Student ID: _____ Name: _____

Signature: _____ Date: _____

The exam is closed-book and closed-notes. Read the questions carefully and focus your answers on what has been asked. You are allowed to ask the instructor/TAs for help only in understanding the questions, in case you find them not completely clear. Be concise and precise in your answers and state clearly any assumption you may have made. All your answers must be included in the attached sheets. You have 150 minutes to complete your exam. Be wise in managing your time.

Please do not fill in the "Score" fields below. Self-grading is not allowed. Good luck!

1	/15
2	/15
3	/8
4	/8
5	/15
6	/10
7	/10
8	/9
Total	/90

1. Hash Tables and Binary Search Trees

① (1 point for each) True or False

- i. Linear probing uses linked lists to resolve collisions.

Your answer: (True / False) **False**

- ii. In-order traversal of a binary search tree (BST) prints sorted keys.

Your answer: (True / False) **True**

- iii. The time complexity of freeing a binary search tree (BST) is $O(\log n)$ for balanced trees.

Your answer: (True / False) **False**

- ② (3 points) Consider the following hash function used in a hash table. Assuming there are n items stored in the hash table, determine the worst-case time complexities of the following operations: (1) add, (2) search, and (3) free.

```
unsigned int hash(const char *s) {  
    return 0;  
}
```

Your answer:

add: _____

search: _____

free: _____

$O(n)$ or $O(1)$ (1 point)

$O(n)$ (1 point)

$O(n)$ (1 point)

- ③ (2 points) Consider the following function for removing a node with a given key from a hash table that uses separate chaining. Fill in the **blank A** and **blank B** to correctly complete the implementation.

```
enum {BUCKET_COUNT = 1024};  
struct Node {  
    const char *key;  
    int value;  
    struct Node *next;  
};  
struct Table {  
    struct Node *array[BUCKET_COUNT];  
};  
  
int Table_remove(  
    struct Table *t,  
    const char *key)  
{  
    int h;  
    struct Node *p, *prev;  
  
    h = hash(key);  
    /* Assume this hash function  
     * does not produce collisions.  
     */  
    p = t->array[h];  
    prev = NULL;  
  
    while (p != NULL) {  
        if (strcmp(p->key, key) == 0) {  
            if (prev == NULL)  
                blank A = p->next;  
            else  
                prev->next = blank B;  
  
            free((char*)p->key);  
            free(p);  
            return 1;  
        }  
  
        prev = p;  
        p = p->next;  
    }  
  
    return 0;  
}
```

Your answer:

blank A: _____

blank B: _____

A: $t \rightarrow \text{array}[h]$, B: $p \rightarrow \text{next}$ (1 point for each)

- ④ (2 points) Explain why the following hash function is a poor hash function.

```
unsigned int hash(const char *x)
{
    unsigned int h = 0;

    while (*x != '\0') {
        h += *x;
        x++;
    }

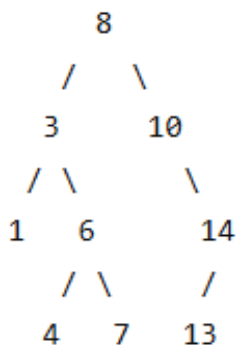
    return h % 1024;
}
```

Your answer:

Only the sum of characters matters. Character order is ignored.

- ⑤ (1 point) Suppose the following keys are inserted into an empty BST (binary search tree) in this order: 8, 3, 10, 1, 6, 14, 4, 7, 13. Draw the final BST.

Your answer:



- ⑥ (2 points) Consider the following function that finds the k-th smallest key in a binary search tree (BST). Fill in the blank A and blank B to correctly complete the implementation.

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int key;
    int size;
    /* The size field satisfies
    node->size = 1 + size(left
    subtree) + size(right subtree).
    */
    struct Node *left;
    struct Node *right;
};

int get_size(struct Node *node) {
    if (node == NULL)
        return 0;
    return node->size;
}

/* Return the k-th smallest key */
int kth_smallest(
    struct Node *root, int k) {

    if (root == NULL)
        return -1; // invalid

    int left_size =
        get_size(root->left);

    if (k == left_size + 1)
        return root->key;
    else if (blank A)
        return kth_smallest(
            root->left, k);
    else
        return kth_smallest(
            root->right, blank B);
}
```

Your answer:

blank A: _____

blank B: _____

A: $k \leq \text{left_size}$ or $k < \text{left_size} + 1$,

B: $k - \text{left_size} - 1$ (1 point for each)

- ⑦ (2 points) Consider the following function that removes a node having two children. Fill in the blank A and blank B to correctly complete the implementation.

```
struct Node* Tree_remove_node(  
    struct Node *root, int key) {  
  
    ...  
  
    } else {  
        ... // cases 1 and 2  
  
        struct Node *x =  
            find_max(root->blank A);  
        /* finds the node with the  
        largest key */  
  
        root->key = x->key;  
        root->value = x->value;  
        root->left = Tree_remove_node(  
            blank B, x->key);  
    }  
  
    return root;  
}
```

Your answer:

blank A: _____

blank B: _____

A: left, B: root->left (1 point for each)

- ② (2 points) Translate the following assembly code into equivalent C code.

```
mov $0, %rax  
loop:  
cmp $0, %rdi  
jle end  
add %rdi, %rax  
sub $1, %rdi  
jmp loop  
end:
```

Your answer:

```
long sum = 0;  
  
while (n > 0) {  
    sum += n;  
    n--;  
}  
All the other logically equivalent  
codes are correct.
```

2. Assembly Overview and Instructions

- ① (1 point for each) True or False
- i. Assembly language is portable across different CPU architectures.

Your answer: (True / False) **False**

- ii. The instruction `cmp $1, %rdx` stores the subtraction result in `%rdx`.

Your answer: (True / False) **False**

- ③ (1 point) Consider the following assembly code. Assume %rdx = 7 initially. What is the final value stored in %rcx?

```
mov $0, %rcx
loop:
cmp $1, %rdx
jle endloop
add $1, %rcx
mov %rdx, %rax
and $1, %rax
je even
mov %rdx, %rax
add %rax, %rdx
add %rax, %rdx
add $1, %rdx
jmp loop
even:
sar $1, %rdx
jmp loop
endloop:
```

Your answer: _____

16

- ④ (3 points) Consider the following C code. Identify where each variable is stored in memory: y, z, *p

```
int x = 5;
static int y;
const int z = 10;

int main() {
    int a;
    int *p = malloc(sizeof(int));
}
```

Your answer:

y: _____

(DATA / BSS / RODATA / STACK / HEAP)

z: _____

(DATA / BSS / RODATA / STACK / HEAP)

*p: _____

(DATA / BSS / RODATA / STACK / HEAP)

y: BSS, z: RODATA, *p: HEAP (1 point for each)

- ⑤ (1 point) Consider this assembly code: "movb \$0xAB, %al". Which bits of %rax are modified?

Your answer:

Only the lowest 8 bits of %rax are modified.

- ⑥ (1 point) Consider this assembly code: "mov 24(%rbx,%rcx,8), %rax".

If %rbx = 1000 and %rcx = 5, what effective memory address is accessed by this instruction?

Your answer: _____

1064

- ⑦ (3 points) Consider the following assembly code. Determine the value of ZF(Zero Flag), CF(Carry Flag), and OF(Overflow Flag).

```
movb $127, %al
addb $1, %al
```

Your answer:

ZF: _____

CF: _____

OF: _____

ZF = 0 CF = 0 OF = 1 (1 point for each)

- ⑧ (2 points) Consider the C code shown on the left. Assuming %rax stores i, %rbx stores sum, and %rcx is available as a temporary register, fill in the blank A and blank B in the assembly code on the right to correctly implement the C code.

```
long A[10];
long sum = 0;
for (i = 0; i < 10; i++)
    sum += A[i];
```

```
mov $0, %rax
mov $0, %rbx
Loop:
blank A
blank B
inc %rax
cmp $9, %rax
jle Loop
```

Your answer:

blank A: _____

blank B: _____

A: `mov A(%rax,8), %rcx`

B: `add %rcx, %rbx` (1 point for each)

3. Assembly Function Calls

- ① (1 point for each) True or False
- i. A function may freely overwrite %rbx without restoring it.
Your answer: (True / False) **False**
- ii. The caller is responsible for removing stack-passed arguments after the call returns.
Your answer: (True / False) **True**

- ② (1 point) Consider the following assembly code. What is the final value of %rsp?

```
movq $0x1000, %rsp
pushq %rax
pushq %rbx
popq %rcx
```

Your answer: _____

0x0ff8

- ③ (1 point) Suppose a function executes the following code. What is the offset of the local variable from %rbp?

```
pushq %rbp
movq %rsp, %rbp
pushq %rbx
pushq %r12
pushq %r13
subq $8, %rsp
```

Your answer: _____

-32(%rbp) or -32

- ④ (2 points) Consider the following assembly code. What are the final values of %rbx and %rcx?

```
movq $100, %rax
pushq %rax
movq $200, %rax
pushq %rax
popq %rbx
popq %rcx
```

Your answer:

%rbx: _____

%rcx: _____

- ⑤ (1 point) Consider the following execution sequence. Assume execution reaches the body of R() when called from Q(). The stack currently contains two return addresses. Which return address is on the top of the stack?

<pre>P() { R(); Q(); }</pre>	<pre>Q() { R(); }</pre>
--------------------------------------	-----------------------------

Your answer: _____

The return address back to Q() is on top of the stack.

- ⑥ (2 points) Fill in the blank of the function foo().

```
foo:
    pushq %rbp
    movq %rsp, %rbp
    subq $16, %rsp
    ...
    blank
    popq %rbp
    ret
```

Your answer: _____

movq %rbp, %rsp

4. Processes and Exceptions

- ① (2 points) Classify each event.

- i. User presses a key

Your answer: _____

(Interrupt / Trap / Fault / Abort)

Interrupt (1 point)

- ii. Page not in memory

Your answer: _____

(Interrupt / Trap / Fault / Abort)

Fault (1 point)

- ② (1 point) Which of the following are asynchronous?

- i. Divide-by-zero
- ii. Keyboard input
- iii. Page fault
- iv. Disk controller completion

Your answer: _____

ii. Keyboard input and iv. Disk controller completion

(1 point only when both answers are correct)

- ③ (1 point for each) True or False

- i. A trap is asynchronous because it transfers control to the OS.

Your answer: (True / False) False

- ii. A page fault always terminates the program.

Your answer: (True / False) False

- iii. An exception handler executes with more privileges than ordinary application code.

Your answer: (True / False) True

- ④ (1 point) Fill in the `blank` assembly instruction that causes a trap to the operating system.

```
movq $9, %rax
movq $0, %rdi
movq $4096, %rsi
blank
```

Your answer: _____

`syscall`

- ⑤ (1 point) A process has requested keyboard input and is waiting for the user to type. Which state is the process in?

- i. Ready
- ii. Running
- iii. Waiting
- iv. Terminated

Your answer: _____

`iii. Waiting`

5. Memory Management

- ① (1 point for each) True or False

- i. Two different virtual pages may map to the same physical page.

Your answer: (True / False) `True`

- ii. Internal fragmentation occurs when free memory exists but is not contiguous.

Your answer: (True / False) `False`

- iii. Reference counting can reclaim cyclic garbage.

Your answer: (True / False) `False`

- ② (2 points) Consider the following code. Identify all forms of (1) temporal and (2) spatial locality exhibited by this code.

```
for (i = 0; i < n; i++) {
    sum += a[i];
}
```

Your answer:

`Temporal locality`

`Strong temporal locality:`

- `a[0]` is repeatedly accessed.
- `sum` is repeatedly accessed.
- Loop instructions are repeatedly executed.

`Spatial locality`

`Very weak spatial locality:`

- Only `a[0]` is accessed.
- Nearby elements such as `a[1]`, `a[2]` are never referenced.

`(any one of these for temporal locality`

`→ 1 point)`

`(any one of these for spatial locality → 1 point)`

③ (1 point for each) A machine uses 32-bit virtual addresses and 4 KB pages.

i. How many offset bits exist?

Your answer: _____

12 bits

ii. How many virtual pages exist?

Your answer: _____

2^{20} pages

④ (1 point) Suppose the page size is 4096 bytes, virtual page 7 maps to physical page 25, and a process accesses virtual address $7 \times 4096 + 100$; what physical address is generated?

Your answer: _____

102500

⑤ (1 point) Given a page size of 4096 bytes and the virtual address 0x12345ABC, which belongs to virtual page 0x12345, what is the offset within the page?

Your answer: _____

0xABC (2748 in decimal)

⑥ (2 points) Given an initially free heap consisting of a single 256-byte block, assuming first-fit allocation, block splitting when necessary, and coalescing of adjacent free blocks upon deallocation, what is the final state of the heap after the following sequence of memory allocation and deallocation operations?

```
p1 = malloc(64);  
p2 = malloc(32);  
p3 = malloc(48);  
free(p2);  
p4 = malloc(24);  
free(p1); free(p3); free(p4);
```

Your answer: 256 free

⑦ (2 points) Suppose the current program break is 0x50000000; after executing the following code, determine the values stored in p1 and p3?

```
void *p1 = sbrk(0);  
void *p2 = sbrk(4096);  
void *p3 = sbrk(2048);
```

Your answer:

p1 = _____

0x50000000 (1 point)

p3 = _____

0x50001000 (1 point)

- ⑧ (2 points) Consider the following code; after the final statement has executed, which dynamically allocated objects are considered garbage?

```
Node *A = malloc(sizeof(Node));
Node *B = malloc(sizeof(Node));
Node *C = malloc(sizeof(Node));
A->next = B;
B->next = C;
A = NULL;
```

Your answer: _____

A, B, and C

(2 points only when all answers are correct)

- ② (1 point) Consider the following code. What may happen?

```
pipe(p);
if (fork() == 0) {
    close(p[1]);
    while (read(p[0], &c, 1) > 0)
        ;
} else {
    write(p[1], "ABC", 3);
    wait(NULL);
}
```

Your answer: The child may block forever.

6. I/O Management

- ① (1 point for each) True or False
- i. A FILE * is implemented directly by the operating system.

Your answer: (True / False) False

- ii. Pipes provide bidirectional communication through a single descriptor.

Your answer: (True / False) False

- ③ (1 point) Suppose a file contains 0123456789. Which characters are read during the second read() call?

```
int fd = open("f.txt", O_RDONLY, 0);
char buf[4];
read(fd, buf, 3);
lseek(fd, 2, SEEK_CUR);
read(fd, buf, 2);
```

Your answer: _____

56

- ④ (2 points) Given the implementation of `myfgetc()`, fill in the `blank A` and `blank B` to correctly complete the function. `myfgetc()` reads and returns the next character from a specified file stream, advancing the file position by one character.

```
int myfgetc(FILE *fp) {
    if (blank A >= 0)
        return (unsigned char) blank B ;

    fp->cnt =
        read(fp->fd, fp->base, 1024);
    if (fp->cnt <= 0)
        return EOF;

    fp->ptr = fp->base;
    fp->cnt--;
    return (unsigned char) blank B ;
}
```

Your answer:

blank A: _____

blank B: _____

A: `--fp->cnt` (1 point)

B: `*fp->ptr++` (1 point)

- ⑤ (2 points) Assume the current directory contains the following files.

mail1 From: A Content-Type: text/plain Content-Type: image/jpeg
mail2 Content-Type: text/plain Subject: Hello
mail3 content-type: image/jpeg Content-Type: application/pdf

What is the exact output of the following command?

```
grep -i Content-Type * | cut -d" "
-f2 | sort | uniq -c | sort -nr
```

Your answer:

2 text/plain

2 image/jpeg

1 application/pdf

- ⑥ (1 point) Consider the following C code, which is compiled into 'a.out'.

```
printf("Start");
sleep(60);
```

Assume the output is redirected on the shell as follows. After 10 seconds, what is observed in `out.txt`?

```
./a.out > out.txt
```

Your answer: Nothing appears (`out.txt` is empty)

- ⑦ (1 point) Suppose the file "data.bin" contains exactly 300 bytes. What is the value of r3?

```
char buf[500];
int fd =
    open("data.bin", O_RDONLY, 0);
int r1 = read(fd, buf, 200);
int r2 = read(fd, buf, 200);
int r3 = read(fd, buf, 200);
```

Your answer: _____

0

- ② (1 point) Consider the following program. How many processes exist?

```
int main(void) {
    if (fork() == 0)
        fork();
    fork();
    return 0;
}
```

Your answer: _____

6 processes

7. Process Management

- ① (1 point for each) True or False

- i. A child process created by fork() shares the same address space with its parent, so modifications to a variable in the child are immediately visible to the parent.
Your answer: (True / False) **False**
- ii. The statement pid = fork(); is executed once but returns twice.
Your answer: (True / False) **True**
- iii. If a child reads one byte from a shared open file descriptor after fork(), the parent's file offset is unaffected.
Your answer: (True / False) **False**

- ③ (1 point) Consider the following code. Under what conditions is "B" printed?

```
int main(void) {
    printf("A\n");
    char *argv[] = {"ls", NULL};
    execvp("ls", argv);
    printf("B\n");
    return 0;
}
```

Your answer:

When "execvp("ls", argv);" fails

- ④ (1 point) Consider the following code, where the file "data.txt" contains "ABCDE". What character does the parent process print?

```
int fd = open("data.txt", O_RDONLY);
if (fork() == 0) {
    char c;
    read(fd, &c, 1);
    exit(0);
}
wait(NULL);
char c;
read(fd, &c, 1);
printf("%c\n", c);
```

Your answer: _____

B

- ⑤ (2 points) Assume the file descriptors 0,1,2 are open and execute the following code. What are fd1, fd2, and fd3?

```
fd1 = open("A", O_RDONLY);
fd2 = open("B", O_RDONLY);
close(fd1);
fd3 = dup(fd2);
```

Your answer:

fd1: _____

fd2: _____

fd3: _____

fd1=3, fd2=4, fd3=3

(2 points only when all three answers are correct)

- ⑥ (2 points) The following function run_pipe() implements a shell that supports a single pipe: cmd1 | cmd2. Fill in the blank A, blank B, blank C and blank D.

```
/* argv1 = args for cmd1,
   argv2 = args for cmd2 */
void run_pipe(
    char **argv1, char **argv2) {

    int pfd[2];
    if (pipe(pfd) < 0) { exit(1); }

    pid_t p1 = fork();
    if (p1 == 0) {
        blank A ; /* unused read end */
        close(1);
        blank B ;
        /* stdout = write end */
        close(pfd[1]);
        execvp(argv1[0], argv1);
        exit(EXIT_FAILURE);
    }

    pid_t p2 = fork();
    if (p2 == 0) {
        blank C ; /* unused write end */
        close(0);
        blank D ; /* stdin = read end */
        close(pfd[0]);
        execvp(argv2[0], argv2);
        exit(EXIT_FAILURE);
    }

    close(pfd[0]);
    close(pfd[1]);
    waitpid(p1, NULL, 0);
    waitpid(p2, NULL, 0);
}
```

Your answer:

blank A: _____

blank B: _____

blank C: _____

blank D: _____

close(pfd[0]) (0.5 point)

dup(pfd[1]) or dup2(pfd[1], 1) or fcntl(pfd[1], F_DUPFD, 1) (0.5 point)

close(pfd[1]) (0.5 point)

dup(pfd[0]) or dup2(pfd[0], 0) or fcntl(pfd[0], F_DUPFD, 0) (0.5 point)

8. Signals

- ① (3 points) Consider the following sequence of shell commands, where "infloop" is an infinite loop.

```
$ ./infloop
Ctrl-Z
$ bg
$ kill -20 <pid>
$ fg
Ctrl-C
```

The following table shows (1) the signal sent, (2) whether the signal originates from an interrupt or a trap, and (3) the resulting process state for each step. Please fill in the blank A, blank B, and blank C.

Action	Signal	Interrupt/ Trap	Resulting State
Run program	None	N/A	Running foreground
Ctrl-Z	SIGTSTP	blank B	Stopped
bg	blank A	Trap	Running background
kill -20 pid	SIGTSTP	Trap	Stopped
fg	SIGCONT	Trap	Running foreground
Ctrl-C	blank C	Interrupt	Terminated

Your answer:

blank A: _____

blank B: _____

blank C: _____

SIGCONT (1 point)

Interrupt (1 point)

SIGINT (1 point)

- ② Consider the following program.

```
void myHandler(int sig) {
    printf("caught signal %d\n", sig);
}

int main(void) {
    if (signal(SIGKILL, myHandler)
        == SIG_ERR)
        printf("signal registration
failed\n");
    else
        printf("signal registration
succeeded\n");

    printf("OK\n");

    while (1) {
        sleep(1);
    }
    return 0;
}
```

(1 point) What output will be printed?

Your answer:

signal registration failed

OK (1 point)

(1 point) Assume the program enters the while loop. If the command "kill -9 <pid>" is executed, what output will be printed? Assume <pid> is the process ID of the program.

Your answer: Nothing will be printed (1 point)

- ③ (1 point) Consider the following program. What output will be printed?

```
int count = 0;

void h(int sig) {
    count++;
}

int main(void) {
    sigset_t set;
    signal(SIGINT, h);
    sigemptyset(&set);
    sigaddset(&set, SIGINT);
    sigprocmask(SIG_BLOCK, &set, NULL);
    raise(SIGINT);
    raise(SIGINT);
    count++;
    sigprocmask(SIG_UNBLOCK, &set,
        NULL);
    printf("%d\n", count);
}
```

Your answer: _____

2

- ④ (3 points) We want to write code that terminates the program if the user does not enter a number within five seconds. Please fill in the blank A, blank B, and blank C.

```
static void timeout(int sig) {
    printf("Timeout\n");
    exit(EXIT_FAILURE);
}

int main(void) {
    int x;
    sigset_t set;
    sigemptyset(&set);

    blank A ;

    sigprocmask(SIG_UNBLOCK,
        &set, NULL);

    blank B ;

    printf("Enter value: ");
    alarm(5);
    scanf("%d", &x);

    blank C ;

    return 0;
}
```

Your answer:

blank A: _____

blank B: _____

blank C: _____

sigaddset(&set, SIGALRM) (1 point)

signal(SIGALRM, timeout) (1 point)

alarm(0) (1 point)